

PhantomGrid

AI-Driven Passive Behavioural Authentication Engine for Public Sector Banks

Team ZeroIntent (S.No. 8) | IIIT Kottayam | Madapati Jyoti Radithya, Kontheti Sai Akhilesh, Praising Y Harris Ratnam

1. Problem Statement

Indian Public Sector Banks lost over INR 7,400 crore to digital fraud in FY2023, the majority occurring AFTER successful login. Current authentication verifies identity once at the login gate, then trusts the session unconditionally. An attacker with stolen credentials (phishing, SIM swap, data breach) is indistinguishable from the legitimate user by any existing control. PhantomGrid answers the question no system asks at the point of a fund transfer: is the person operating this session the enrolled account holder? It answers continuously, on every transaction, with zero friction for genuine users.

2. Proposed Solution

A three-layer passive behavioural authentication engine that runs silently beneath a banking portal. It observes unconscious behavioural signatures - decoy interaction, navigation intent, and PIN keystroke rhythm - that are stable over time yet practically impossible to replicate. Enrollment (sessions 1-5) silently learns the user baseline; from session 6 every transaction is scored 0-100 and mapped to a decision: ALLOW (<60, zero friction), OTP step-up (60-79), or BLOCK (>=80). Even with the correct PIN, an attacker types in a different rhythm and is blocked. The killer property: you can steal a password, but you cannot steal a rhythm.

3. System Architecture

TIER 1	Signal Capture (Frontend)	NexaBank portal + capture.js behavioural hooks onDecoyTap / onBeneDwell / onAmountKey / onPinKey / onPaySubmit -> one JSON package
TIER 2	ML Inference (FastAPI + Python 3.12 + SQLAlchemy + SQLite)	L1 IsolationForest L2 IsolationForest L3 Dynamic Time Warping Continuous scoring -> Fusion ($L1*0.30 + L2*0.40 + L3*0.30$) -> ALLOW/OTP/BLOCK SHA-256 replay defence + tamper-evident hash-chain audit log
TIER 3	Analyst Dashboard (Vanilla JS)	polls GET /logs every 2s; gauge, layer bars, trend chart, attack heatmap, RBI panel, live QR, audit badge, maturity indicator

4. Technology Stack

Component	Technology	Rationale
API framework	FastAPI (Python 3.12) + Uvicorn	Async, auto-validation, Swagger UI
Anomaly detection (L1,L2)	scikit-learn IsolationForest	Unsupervised - no fraud labels at enrollment
Rhythm comparison (L3)	dtadistance (DTW)	Tolerates natural keystroke speed variation
Database / ORM	SQLite + SQLAlchemy 2.0	Zero-config PoC; PostgreSQL-compatible schema
Validation / Crypto	Pydantic v2 / hashlib SHA-256	422 on bad input; replay + audit chain
Frontend / Dashboard	Vanilla HTML/CSS/JS	No build step, fully portable
Deployment	Railway (Nixpacks)	Live: phantomgrid-production.up.railway.app

5. Workflow

Enrollment: capture.js collects signals during a normal transaction and POSTs to /enroll; after 5 samples the baseline is complete and capture.js auto-switches to verification. Verification: each /verify call runs a replay check (SHA-256 signature, 5-min window), scores all three layers, fuses them, applies adaptive learning on ALLOW, writes a hash-chained session log, and returns the decision.

```
POST /verify {user_id, signals}
1 replay check SHA-256(payload) seen in 5 min ? yes -> forced BLOCK
2 L1 = IF([decoy_taps, hesitations]) L2 = IF([bene_dwell, avg_iki])
  L3 = DTW(pin_vector vs enrolled baseline, best match)
3 composite = L1*0.30 + L2*0.40 + L3*0.30
4 decision = ALLOW(<60) | OTP(60-79) | BLOCK(>=80)
5 if ALLOW: append session to baseline (sliding window 20)
6 row_hash = SHA256(prev_hash | session fields) ; write session_logs ; return decision
```

6. Database Design

Two SQLite tables. Baselines are stored as JSON vectors and the IsolationForest is fit on-the-fly per request (no model serialization), so the model always reflects the latest adaptive baseline.

Table	Key Columns	Purpose
user_profiles	user_id (unique), layer1_vectors, layer2_vectors, pin_vectors (JSON)	Per-user behavioural baseline
session_logs	session_id, timestamp, user_id, layer1/2/3_score, composite_score, decision, row_hash, prev_hash	Scored sessions + tamper-evident audit chain

7. APIs and External Services

No third-party APIs, cloud ML, or external data providers - all computation is local. Core REST endpoints:

Endpoint	Method	Description
/enroll	POST	Store one enrollment sample (5 to complete baseline)
/verify	POST	Score a session -> layer scores, composite, decision, replay_detected
/logs?user_id=	GET	Recent session rows (optional user filter) - drives the dashboard
/maturity?user_id=	GET	Baseline maturity / confidence (cold-start indicator)
/audit/verify	GET	Walk hash chain -> {valid, broken_at_session}
/reset_user	POST	Clear a profile for clean re-enrollment (demo support)

8. AI / ML Models

Isolation Forest (Liu et al., 2008) - unsupervised anomaly detection for L1 and L2. At enrollment only legitimate sessions exist, so a supervised classifier is infeasible; IF learns the per-user normal and flags deviation. On small (5-sample) baselines the raw anomaly score saturates, so we add a continuous normalized-deviation magnitude (services/scoring.py): inliers score 0-45, outliers 55-100; below 10 samples pure deviation is used for determinism.

Dynamic Time Warping - L3 RhythmLock compares PIN inter-keystroke intervals against the baseline. DTW aligns sequences elastically before measuring distance, tolerating natural speed variation (a rushed user typing 10% faster is still recognised), reducing the false rejection rate versus Euclidean distance. Distance maps smoothly to 0-100.

Validation: a 200-session synthetic benchmark gives 96.7% detection rate, 0.0% false positive rate, AUC 1.00, inference < 5ms.

9. Security Measures

- Replay defence: every /verify payload is SHA-256 signed; an exact duplicate within 5 minutes is forced to BLOCK (replay_detected=true).
- Tamper-evident audit: session_logs is a SHA-256 hash chain (row_hash links to prev_hash); any edit is caught by GET /audit/verify.
- PII minimisation: PIN digits are never stored - only millisecond inter-keystroke intervals, which cannot reconstruct the PIN.
- Input validation: Pydantic v2 schemas reject malformed requests (HTTP 422); SQLAlchemy ORM removes SQL-injection surface.
- STRIDE coverage: Spoofing (per-user models), Tampering (hash chain), Repudiation (immutable logs), Info Disclosure (no PII) mitigated; DoS and Elevation-of-Privilege addressed in the production roadmap (rate-limiting, authenticated /enroll).

10. Scalability Considerations

Concern	PoC	Production Path
Database	SQLite single file	PostgreSQL - drop-in connection string change
Model storage	Fit on-the-fly	Pre-serialised per-user models (joblib) + Redis cache
Dashboard	REST polling 2s	WebSocket streaming at 50ms
Deployment	Railway single region	Docker + AWS ap-south-1 (RBI data localisation)
Latency	< 5ms per session	Unchanged - runs parallel to payment, never blocks it

11. Assumptions

- Users enroll on the same device used for live transactions (keystroke rhythm is device-specific).
- At least 5 enrollment samples exist before scoring is enforced (maturity endpoint signals readiness).
- /enroll is called within an authenticated banking session in production (prevents enrollment poisoning).

12. Limitations

- Cold start: below 5 samples the model is undertrained; the maturity indicator flags low confidence and production uses shadow mode.
- Cross-device: PIN rhythm shifts across keyboards; mitigation is per-device baseline profiles (roadmap).
- Input channel: keystroke dynamics need a physical keyboard - on-screen mouse-numpad timing is too noisy to separate users, so the PoC captures PIN rhythm from keyboard input.
- Sustained adaptive poisoning over many sessions remains a residual risk; mitigation is drift detection (roadmap).

13. Future Enhancements

Enhancement	Description	Priority
WebAuthn biometric step-up	On amber (60-79), trigger a WebAuthn / fingerprint (Windows Hello, Face ID) challenge BEFORE OTP. The biometric never leaves the device, is phishing- and SIM-swap-resistant, and an attacker on another device cannot produce it. OTP remains the fallback for users without biometric hardware.	High
Device-aware profiles	Separate baseline per device fingerprint - removes cross-device false rejections.	High
PostgreSQL + Redis	Horizontal scaling and pre-cached per-user models for sub-ms inference.	High
Mouse / touch dynamics (L4)	Scroll velocity, touch pressure, pointer-trajectory entropy for mobile banking.	Medium
WebSocket streaming	Real-time dashboard at 50ms instead of 2s polling.	Medium
Drift detection	Statistical test on baseline evolution to detect gradual poisoning.	Medium
RBI reporting module	Per-user risk history and audit export for regulators.	Medium

PhantomGrid - Passive. Invisible. Unbeatable. Live API: <https://phantomgrid-production.up.railway.app> GitHub: github.com/Pyhroff/PhantomGrid (private, shared with CBIHack26)